



main

[Ansible-For-DevOps-Engineers](#) / [Section 02: Ad-Hoc Commands & Inventory](#) / [Labs](#)
/ [Guided Lab 03: Running Your First Ansible Ad-Hoc Commands.md](#)**pravinmishraaws** Create Guided Lab 03: Running Your First Ansible Ad-Hoc Commands.md

bb4d7fa · last month



200 lines (130 loc) · 4.91 KB

Preview

Code

Blame



Raw



Guided Lab: Running Your First Ansible Ad-Hoc Commands

Lab Objective

In this lab, you will:

- Use Ansible's command-line interface to send ad-hoc commands
- Test connectivity with the `ping` module
- Run real-world tasks like checking uptime, installing packages, and restarting services
- Understand how to target specific groups from your inventory
- Use `--become` for elevated privileges

Lab Prerequisites

Before you begin, make sure:

- You have completed Lecture 02 and created a custom **inventory.ini** file
- Your inventory includes at least one Azure VM reachable via SSH
- You have tested SSH access and Ansible `ping` in previous labs
- Your managed VM has `Python` and `apt` package manager (Ubuntu)

Step 1: Confirm Your Inventory File

If you followed the previous lab, you should already have a file like this in your working directory:

```
[web]
<VM_PUBLIC_IP_1>

[db]
<VM_PUBLIC_IP_2>

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
```



If you only have one VM, you can reuse the same IP under both `[web]` and `[db]` for now.

Step 2: Check Connectivity with the `ping` Module

Let's start with the simplest and most important check:

```
ansible all -i inventory.ini -m ping
```



Expected output:

```
<IP> | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```



If this fails:

- Ensure SSH key access is working
- Check that the VM's IP and user are correct in the inventory
- Confirm port 22 is open on the VM

Step 3: Run a Shell Command (Check Uptime)

Let's run a real Linux command remotely using the `shell` module:

```
ansible all -i inventory.ini -m shell -a "uptime"
```



Expected output will vary, but should return the uptime of each host.

Step 4: Install a Package Using the `apt` Module

This time, let's install a package like `nginx` using the `apt` module.

Because this is a privileged operation, we'll add `--become` :

```
ansible web -i inventory.ini -m apt -a "name=nginx state=present" --become
```



What's happening here:

- We're targeting the `web` group
- Using the `apt` module to install a package
- `state=present` ensures it's installed (idempotent)
- `--become` allows Ansible to use `sudo` behind the scenes

You may be prompted for your **local sudo password** if passwordless sudo isn't set up.

Step 5: Restart a Service

Now let's restart a service (e.g., `nginx`) using the `service` module:

```
ansible web -i inventory.ini -m service -a "name=nginx state=restarted"
```



You can also test with:

```
ansible web -i inventory.ini -m shell -a "systemctl status nginx" --become
```

This checks whether the service is active.

Step 6: Target a Specific Group or Host

Let's say you only want to run a command on the `db` group:

```
ansible db -i inventory.ini -m shell -a "df -h"
```

Or a single host:

```
ansible all -i inventory.ini -m shell -a "whoami" --limit <VM_PUBLIC_IP>
```

The `--limit` flag is helpful when you want to filter execution to a specific host or group.

Step 7: Run Multiple Commands (Bonus)

Try a few more useful commands:

```
ansible all -i inventory.ini -m shell -a "hostname"
ansible web -i inventory.ini -m shell -a "free -m"
ansible db -i inventory.ini -m command -a "whoami"
```

The `shell` module supports complex shell commands, while the `command` module is safer for single commands without shell expansion.

Best Practices Recap

- Use **ad-hoc commands** for one-time tasks, updates, or experiments
- Always confirm SSH works before running Ansible commands
- Use `--become` when performing tasks that require `sudo`

- Keep your **inventory file project-specific** and versioned in Git
- Use groups (`[web]` , `[db]`) to structure and reuse commands across environments

Lab Completed

You've now:

- Run your first real ad-hoc commands with Ansible
- Installed and restarted services on remote servers
- Used modules like `ping` , `apt` , `shell` , and `service`
- Scoped commands to groups or individual servers
- Practiced privilege escalation with `--become`

That's real-world automation — and it only gets better from here.

Student Assignment: Build Your Own Ad-Hoc Task Set

Objective:

Practice writing and running ad-hoc commands based on different real-world scenarios.

Tasks:

1. Create and document 5 ad-hoc commands that:
 - Check a system metric (like disk usage, memory, CPU)
 - Install or update a package
 - Start or stop a service
 - Use `--limit` to target one specific host
 - Use a non-default module (`copy` , `setup` , or `command`)
2. Run all your commands on the `web` and `db` groups
3. Record:
 - Your commands
 - The output for each
 - Any errors you faced and how you fixed them

4. (Optional) Try adding a new group to your inventory and test against it